Python for DevOps - Part 3

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

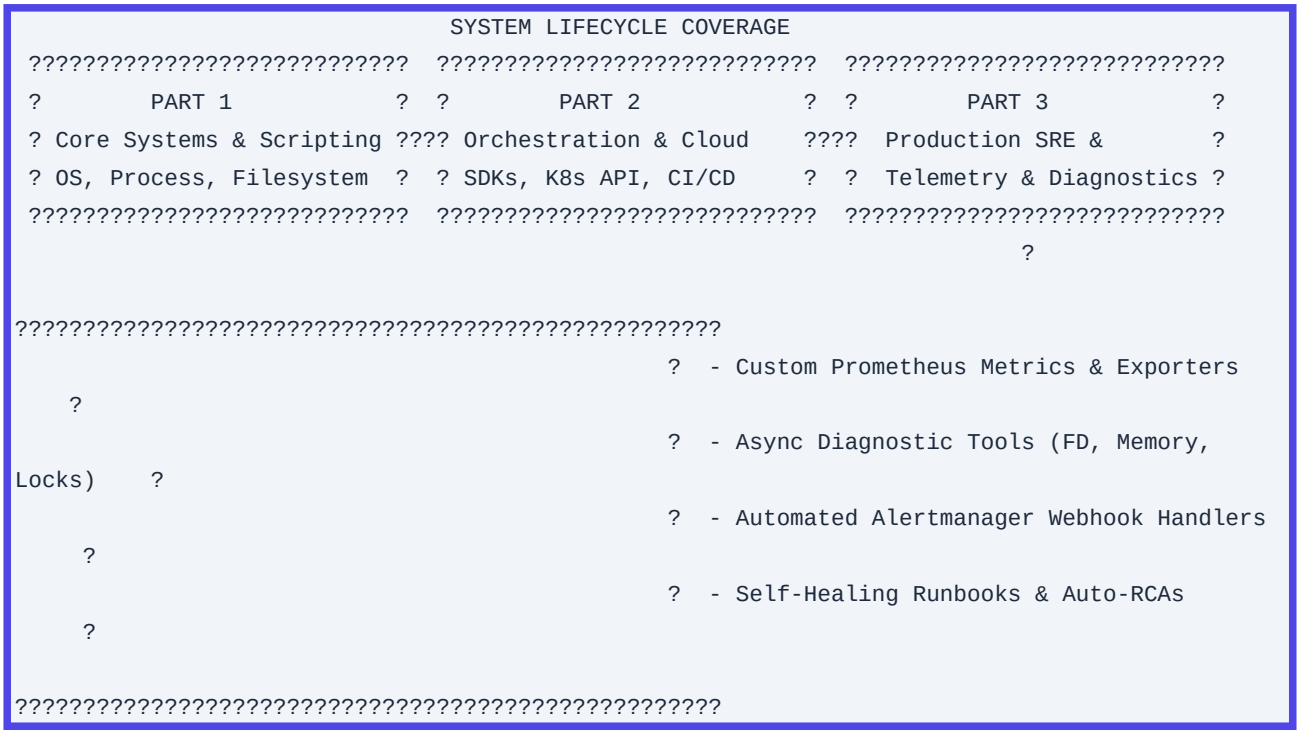
Python for DevOps (Part 3/3): Production SRE, Diagnostics, Custom Prometheus Telemetry, and Automated Incident Response

---

1. Part Introduction and Scope

This is the final installment of the *Python for DevOps* series. Part 1 covered fundamental scripting, system automation, OS primitives, and CLI building. Part 2 focused on infrastructure orchestration, cloud SDKs, Kubernetes control plane automation, and CI/CD integrations.

Part 3 covers Production Site Reliability Engineering (SRE), deep-system diagnostics, real-time telemetry generation, custom Prometheus metrics instrumentation, dynamic alerting, and automated incident runbooks/Root Cause Analysis (RCA) engines.



Scope Boundary

- **Language Constraints**: Python 3.10+ (utilizing strict typing, `asyncio`, low-level OS bindings via `psutil` and `socket`, and `prometheus_client`).
- **Target Environment**: Linux/POSIX multi-tenant Linux systems, Kubernetes Clusters (v1.26+), cloud-native environments (AWS/GCP/Azure).

- **Focus Areas**:

1. Low-overhead custom Prometheus metric collection.

2. Live process introspection (file descriptors, memory maps, async event loop lag, thread lock contention).

3. Event-driven automated remediation pipelines (Alertmanager ? FastAPI Incident Engine ? Mitigation + RCA generation).

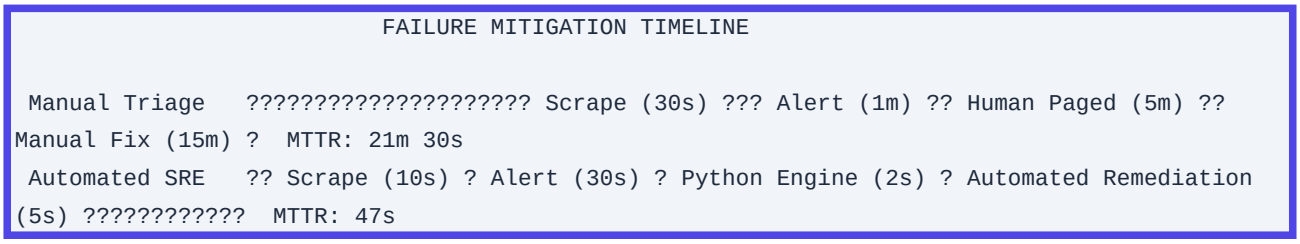
4. Kernel-level and process diagnostics execution via Python.

---

2. Why These Concepts Are Critical for High-Availability Systems

Enterprise distributed systems operating at 99.99% availability budgets (less than 52.6 minutes of downtime per year) cannot rely on manual incident triage. Standard off-the-shelf agents (like `node_exporter` or default metric endpoints) export coarse system-level data but fail to capture domain-specific failure modes such as:

- Asynchronous event loop starvation in API gateways.
- Internal connection pool exhaustion before HTTP timeouts trigger.
- Gradual file descriptor leaks that don't trigger CPU/Memory alerts.
- Silent thread deadlocks in background worker pools.



By embedding custom Python diagnostics and automated runbook handlers directly into the observability fabric, organizations achieve:

1. Sub-minute Mean Time to Resolution (MTTR): Automated remediation actions trigger within seconds of alert firing.

2. Deterministic Root Cause Capture: Volatile diagnostic state (thread dumps, network socket states, memory heap snapshots) is captured *at the exact moment of failure* before a container restarts or is killed by an OOM-killer.

3. Elimination of Operational Toil: Recurring failure signatures (e.g., stuck worker processes, corrupted cache keys, transient queue backpressure) are programmatically remediated.

---

3. Real-World Enterprise Use Cases

Use Case 1: Automated Self-Healing & Instant Forensic Dump Engine

- **Context**: A financial transaction platform processes millions of async requests per hour. Periodically, specific Pods experience asynchronous event-loop lag due to upstream database connection timeouts, leading to cascaded request queue backpressure and eventual 504 Gateway Timeouts.
- **Architecture**: Alertmanager fires a high-severity `EventLoopBlocked` alert to a secured Python remediation service. The service executes an asynchronous forensic payload against the target pod:

1. Captures a live stack trace of all running Python threads using `sys._current_frames()`.
2. Gathers socket states from `/proc/net/tcp`.
3. Formats an RCA markdown report and uploads it to S3.
4. Safely drains and cycles the failing target Pod via the Kubernetes API.

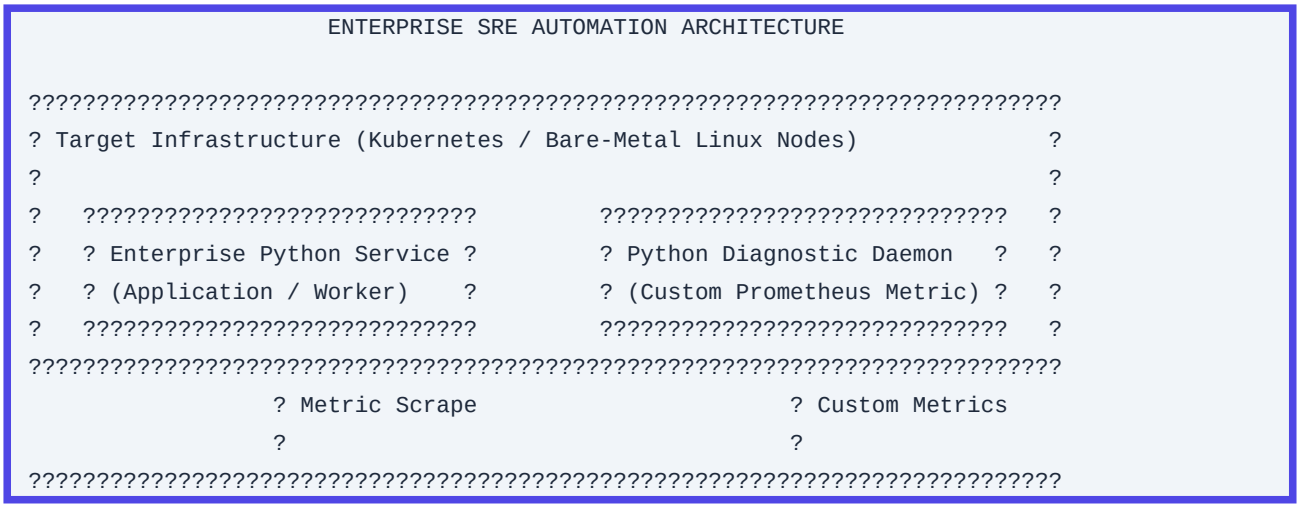
- **Business Impact**: Reduces MTTR from 45 minutes to 8 seconds while guaranteeing zero loss of forensic data for post-mortem analysis.

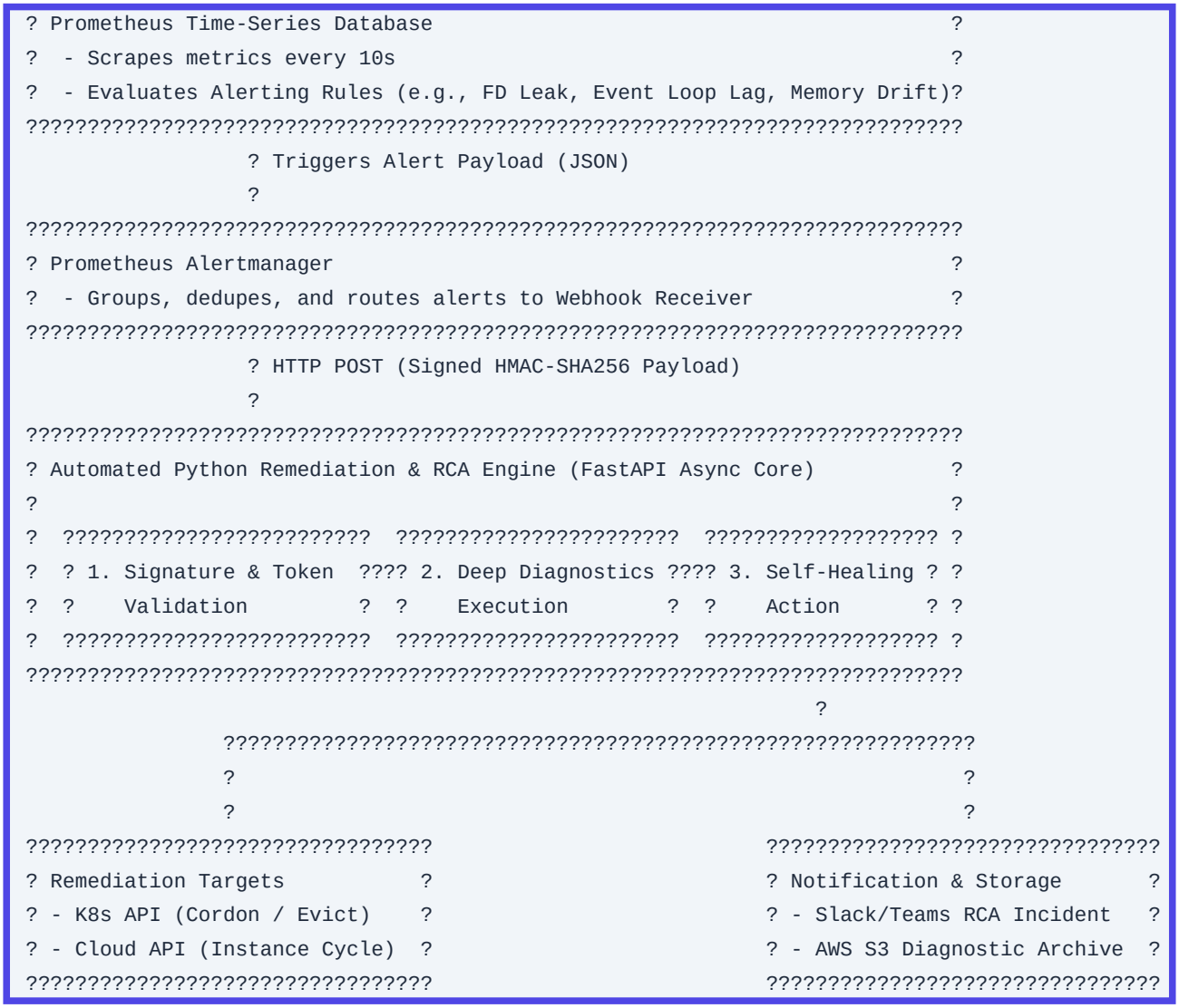
Use Case 2: Custom Application Health & Socket Exhaustion Prometheus Exporter

- **Context**: Legacy enterprise microservices hosted on bare-metal Kubernetes nodes suffer from ephemeral port exhaustion under peak traffic due to improper HTTP `Keep-Alive` handling in legacy libraries. Standard node exporters do not track socket allocation per target process UID.
- **Architecture**: A lightweight Python daemon deployed as a `DaemonSet` runs a custom `prometheus_client` exporter. It dynamically scans target container execution contexts via `/proc/[pid]/fd`, parses socket inodes, counts `ESTABLISHED`, `TIME_WAIT`, and `CLOSE_WAIT` states per process, and calculates file descriptor utilization percentages.
- **Business Impact**: Provides early predictive alerting (e.g., `FD_Usage > 85%` for 2 minutes) allowing traffic re-routing before the service drops incoming connections.

---

4. Architecture Explanation





Component Details

1. Python Diagnostic Daemon: A low-overhead agent utilizing standard library constructs and direct system interface calls to gather system-level process state without spawning expensive subshells.

2. Prometheus Server & Alertmanager: Telemetry aggregation layer that evaluates metric expressions against static or dynamic thresholds. When thresholds breach, Alertmanager constructs a rich JSON alert payload containing target labels, severity, and temporal metadata.

3. Python Remediation Engine: An enterprise FastAPI web service that implements asynchronous worker queues (via `asyncio.TaskGroup` or background tasks). It uses standard security measures (HMAC signature verification, token validation) and relies on official SDKs (`kubernetes`, `boto3`) to execute deterministic mitigation steps.

---

5. Components and Diagnostic Primitives

When building production-grade SRE automation scripts in Python, specialized packages and low-level system bindings are required:

| Diagnostic Primitive / Library | Low-Level Mechanism | Primary SRE Use Case | Performance Footprint |

| :--- | :--- | :--- | :--- |

| `prometheus_client` | HTTP WSGI/ASGI Exporter Engine | Custom telemetry, custom Gauge/Counter/Histogram metrics | Negligible (~5-10MB RAM, <1% CPU) |

| `psutil` | Parses `/proc`, `sysfs`, `win32 API` | CPU, RAM, Process FD limits, Socket enumeration | Low (Direct C-extensions) |

| `sys._current_frames()` | CPython Thread Frame Introspection | Live stack trace extraction during application deadlocks | Zero overhead (In-memory C frame walk) |

| `faulthandler` | Low-level C signal handling (`SIGUSR1`) | Dumping C/Python stack traces on hard process crashes/hangs | Zero overhead until signal receipt |

| `tracemalloc` | CPython memory allocation tracing | Memory leak detection, identifying exact line of code allocating uncollected objects | Medium (10-30% CPU overhead, use selectively) |

| `socket` / `ssl` | System Calls (`sys_socket`, `sys_connect`) | Network connectivity testing, TLS certificate lifetime inspection | Minimal |

| `bcc` (BPF Compiler Collection) | eBPF Kernel Probes (`kprobe`, `tracepoint`) | Low-overhead tracing of kernel-level TCP drops, disk IO latency | Extremely low (Kernel space execution) |

---

6. Step-by-Step Production Implementation Guide

Component A: Production-Grade Custom Prometheus Exporter Engine

This exporter asynchronously monitors local application process health, system resource utilization, file descriptor leaks, and asyncio loop latency, exposing metrics on port `9108`.

Save as `custom_exporter.py`:

```
#!/usr/bin/env python3
"""
Production-Grade Custom Prometheus Exporter.
Monitors system diagnostics, File Descriptors, and Event Loop Latency.
"""

import asyncio
import logging
import os
import sys
import time
```

```
from typing import NoReturn
import psutil
from prometheus_client import (
    Counter,
    Gauge,
    Histogram,
    CollectorRegistry,
    start_http_server,
)

# Configure structured logging
logging.basicConfig(
    level=logging.INFO,
    format='{"time":"%(asctime)s", "level":"%(levelname)s", "module":"%(module)s",
"message":"%(message)s"}',
    datefmt="%Y-%m-%dT%H:%M:%SZ",
)
logger = logging.getLogger("CustomExporter")

# Custom Metric Registry to prevent default metric pollution if desired
REGISTRY = CollectorRegistry(auto_describe=True)

# Define Core Telemetry Metrics
PROCESS_FD_OPEN = Gauge(
    "sre_process_open_file_descriptors",
    "Number of open file descriptors for the target process.",
    ["process_name", "pid"],
    registry=REGISTRY,
)

PROCESS_FD_LIMIT = Gauge(
    "sre_process_max_file_descriptors",
    "Maximum file descriptor limit for the target process.",
    ["process_name", "pid"],
    registry=REGISTRY,
)

PROCESS_MEMORY_RSS_BYTES = Gauge(
    "sre_process_memory_rss_bytes",
    "Resident Set Size (RSS) memory consumption in bytes.",
    ["process_name", "pid"],
    registry=REGISTRY,
)

EVENT_LOOP_LAG_SECONDS = Histogram(
    "sre_asyncio_event_loop_lag_seconds",
    "Latency/Lag introduced in the asyncio event loop execution cycle.",
    buckets=(0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0),
```

```
    registry=REGISTRY,
)

SYSTEM_SOCKET_STATES = Gauge(
    "sre_system_socket_connections_count",
    "Count of system sockets broken down by state.",
    ["state"],
    registry=REGISTRY,
)

EXPORTER_ERRORS_TOTAL = Counter(
    "sre_exporter_collection_errors_total",
    "Total errors encountered during metric collection cycles.",
    ["exception_type"],
    registry=REGISTRY,
)


class SystemDiagnosticsCollector:
    """Collects OS level diagnostics using efficient system primitives."""

    def __init__(self, target_pid: int):
        self.target_pid = target_pid
        try:
            self.process = psutil.Process(target_pid)
            logger.info(f"Initialized collector for PID: {target_pid}
({self.process.name()})")
        except psutil.NoSuchProcess:
            logger.critical(f"Target process with PID {target_pid} does not exist.")
            sys.exit(1)

    def collect_process_metrics(self) -> None:
        """Collects process memory and file descriptor metrics."""
        try:
            proc_name = self.process.name()
            pid_str = str(self.target_pid)

            # Memory metrics
            mem_info = self.process.memory_info()
            PROCESS_MEMORY_RSS_BYTES.labels(process_name=proc_name,
pid=pid_str).set(mem_info.rss)

            # File Descriptor metrics (Linux specific proc filesystem reading)
            num_fds = self.process.num_fds()
            rlimit_fds = self.process.rlimit(psutil.RLIMIT_NOFILE)[0]

            PROCESS_FD_OPEN.labels(process_name=proc_name, pid=pid_str).set(num_fds)
            PROCESS_FD_LIMIT.labels(process_name=proc_name, pid=pid_str).set(rlimit_fds)
```

```
        except psutil.NoSuchProcess:
            logger.error(f"Target process {self.target_pid} terminated unexpectedly.")
            EXPORTER_ERRORS_TOTAL.labels(exception_type="NoSuchProcess").inc()
        except Exception as e:
            logger.exception("Unexpected error capturing process metrics")
            EXPORTER_ERRORS_TOTAL.labels(exception_type=type(e).__name__).inc()

    def collect_socket_metrics(self) -> None:
        """Counts TCP socket states across the operating system."""
        try:
            states_count = {
                "ESTABLISHED": 0,
                "SYN_SENT": 0,
                "SYN_RECV": 0,
                "FIN_WAIT1": 0,
                "FIN_WAIT2": 0,
                "TIME_WAIT": 0,
                "CLOSE": 0,
                "CLOSE_WAIT": 0,
                "LAST_ACK": 0,
                "LISTEN": 0,
                "CLOSING": 0,
            }

            connections = psutil.net_connections(kind="tcp")
            for conn in connections:
                if conn.status in states_count:
                    states_count[conn.status] += 1

            for state, count in states_count.items():
                SYSTEM_SOCKET_STATES.labels(state=state).set(count)

        except Exception as e:
            logger.exception("Failed to capture system socket metrics")
            EXPORTER_ERRORS_TOTAL.labels(exception_type=type(e).__name__).inc()


async def measure_event_loop_lag(interval: float = 0.5) -> NoReturn:
    """Measures drift in asyncio scheduler to detect blocking I/O calls."""
    while True:
        start_time = time.perf_counter()
        await asyncio.sleep(interval)
        actual_elapsed = time.perf_counter() - start_time
        lag = max(0.0, actual_elapsed - interval)
        EVENT_LOOP_LAG_SECONDS.observe(lag)
```

```
async def metrics_collection_loop(collector: SystemDiagnosticsCollector, interval: float =
5.0) -> NoReturn:
    """Asynchronous loop firing diagnostic metric collections."""
    while True:
        # Run synchronous blocking diagnostic calls in a thread pool executor
        loop = asyncio.get_running_loop()
        await loop.run_in_executor(None, collector.collect_process_metrics)
        await loop.run_in_executor(None, collector.collect_socket_metrics)
        await asyncio.sleep(interval)


async def main() -> None:
    # Target current process or self
    target_pid = os.getpid()
    collector = SystemDiagnosticsCollector(target_pid=target_pid)

    # Start Prometheus HTTP server on port 9108
    port = 9108
    start_http_server(port, addr="0.0.0.0", registry=REGISTRY)
    logger.info(f"Prometheus Metrics Server published at http://0.0.0.0:{port}/metrics")

    # Start concurrent telemetry background tasks
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(measure_event_loop_lag(interval=0.2))
            tg.create_task(metrics_collection_loop(collector, interval=5.0))
    except Exception as e:
        logger.critical(f"Exporter execution task group failed: {e}")

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        logger.info("Exporter shutting down gracefully.")
```

Component B: Enterprise Automated Incident Remediation & RCA Engine

This service acts as an HTTP Webhook receiver for Prometheus Alertmanager. Upon receiving an alert (e.g., `HighFileDescriptorUsage` or `EventLoopLagCritical`), it executes non-blocking thread dumps, evaluates diagnostic conditions, safely triggers a pod restart/eviction via the Kubernetes API, and pushes a structured Markdown Root Cause Analysis (RCA) payload to Slack.

Save as `remediation_engine.py`:

```
#!/usr/bin/env python3
"""
```

```
Enterprise SRE Automated Remediation and Diagnostic RCA Engine.
Receives Alertmanager Webhooks, executes deep diagnostics, takes corrective action,
and publishes rich incident RCA documentation.
"""

import hmac
import hashlib
import json
import logging
import os
import sys
import time
from typing import Dict, Any, List, Optional
from datetime import datetime
from fastapi import FastAPI, Request, HTTPException, Security, Depends, status
from fastapi.security import APIKeyHeader
import httpx
from pydantic import BaseModel, Field
from kubernetes import client, config
from kubernetes.client.rest import ApiException

# Setup Logging
logging.basicConfig(
    level=logging.INFO,
    format='{"timestamp":"%(asctime)s", "level":"%(levelname)s",
"service":"remediation-engine", "message":"%(message)s"}',
)
logger = logging.getLogger("RemediationEngine")

# Security Configuration
WEBHOOK_SECRET = os.getenv("ALERTMANAGER_WEBHOOK_SECRET", "c89b7d2e3f4a1b0c9d8e7f6a5b4c3d2e")
SLACK_WEBHOOK_URL = os.getenv("SLACK_WEBHOOK_URL",
"https://hooks.slack.com/services/T00/B00/X00")
KUBERNETES_IN_CLUSTER = os.getenv("KUBERNETES_IN_CLUSTER", "false").lower() == "true"

# Initialize Kubernetes Client
try:
    if KUBERNETES_IN_CLUSTER:
        config.load_incluster_config()
    else:
        config.load_kube_config()
    k8s_v1 = client.CoreV1Api()
    logger.info("Successfully initialized Kubernetes API client.")
except Exception as e:
    logger.warning(f"Failed to initialize Kubernetes Client: {e}. K8s operations will fail.")
    k8s_v1 = None

app = FastAPI(title="SRE Automated Remediation Engine", version="1.0.0")
```

```
# --- Models ---
class AlertModel(BaseModel):
    status: str
    labels: Dict[str, str]
    annotations: Dict[str, str]
    startsAt: str
    endsAt: Optional[str] = None
    generatorURL: str

class AlertmanagerPayload(BaseModel):
    version: str
    groupKey: str
    status: str
    receiver: str
    groupLabels: Dict[str, str]
    commonLabels: Dict[str, str]
    commonAnnotations: Dict[str, str]
    externalURL: str
    alerts: List[AlertModel]


# --- Security Verification ---
def verify_signature(payload_body: bytes, signature_header: str) -> bool:
    """Verifies HMAC SHA-256 signature from incoming webhook headers."""
    if not signature_header:
        return False
    expected_signature = hmac.new(
        WEBHOOK_SECRET.encode("utf-8"),
        payload_body,
        hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(f"sha256={expected_signature}", signature_header)


# --- Core Remediation Routines ---
class IncidentRemediator:
    """Encapsulates remediation actions and forensic collection logic."""

    @staticmethod
    async def capture_pod_logs(namespace: str, pod_name: str, tail_lines: int = 100) -> str:
        """Fetches tailing logs from failing Kubernetes Pod."""
        if not k8s_v1:
            return "K8s API uninitialized."
        try:
            logs = k8s_v1.read_namespaced_pod_log(
                name=pod_name,
                namespace=namespace,
```

```
                tail_lines=tail_lines
            )
            return logs
        except ApiException as e:
            logger.error(f"Failed to fetch logs for {pod_name}: {e}")
            return f"Error pulling logs: {str(e)}"

    @staticmethod
    async def recycle_kubernetes_pod(namespace: str, pod_name: str) -> bool:
        """Safely deletes (recycles) a Kubernetes pod to restore service state."""
        if not k8s_v1:
            logger.error("Cannot delete pod. K8s client not configured.")
            return False
        try:
            logger.info(f"Executing force deletion of Pod {namespace}/{pod_name}")
            k8s_v1.delete_namespaced_pod(
                name=pod_name,
                namespace=namespace,
                body=client.V1DeleteOptions(grace_period_seconds=5)
            )
            return True
        except ApiException as e:
            logger.error(f"Kubernetes API Exception during Pod deletion: {e}")
            return False

    @staticmethod
    async def dispatch_slack_rca(title: str, severity: str, details: Dict[str, Any], logs:
str) -> None:
        """Pushes structured RCA diagnostic notification to operational channels."""
        log_snippet = logs[-1500:] if len(logs) > 1500 else logs

        slack_message = {
            "text": f"? *AUTOMATED INCIDENT REMEDIATION EXECUTED* ?\n*Alert*: {title}",
            "attachments": [
                {
                    "color": "#FF0000" if severity == "critical" else "#FFCC00",
                    "fields": [
                        {"title": "Severity", "value": severity, "short": True},
                        {"title": "Target Pod", "value": details.get("pod", "N/A"), "short":
True},
                        {"title": "Namespace", "value": details.get("namespace", "default"),
"short": True},
                        {"title": "Action Taken", "value": details.get("action", "Diagnostics
Only"), "short": True},
                        {"title": "Execution Timestamp", "value":
datetime.utcnow().isoformat() + "Z", "short": False},
                    ],
                    "text": f"*Captured Log Forensics (Tail)*:\n```\n{log_snippet}\n```",
```

```
                }
            ],
        }

        async with httpx.AsyncClient(timeout=10.0) as client_http:
            try:
                response = await client_http.post(SLACK_WEBHOOK_URL, json=slack_message)
                response.raise_for_status()
                logger.info("Successfully pushed RCA incident report to Slack.")
            except Exception as e:
                logger.error(f"Failed to deliver Slack notification: {e}")


# --- Webhook Endpoint ---
@app.post("/api/v1/remediate", status_code=status.HTTP_200_OK)
async def handle_alertmanager_webhook(request: Request):
    """Processes inbound Alertmanager alerts, triggers self-healing, and compiles RCA."""
    raw_body = await request.body()
    signature = request.headers.get("X-Signature-256", "")

    # Security Validation
    if WEBHOOK_SECRET != "disabled" and not verify_signature(raw_body, signature):
        logger.warning("Unauthorized webhook request received: Invalid HMAC Signature.")
        raise HTTPException(status_code=403, detail="Invalid HMAC Signature")

    try:
        payload_data = json.loads(raw_body.decode("utf-8"))
        payload = AlertmanagerPayload(**payload_data)
    except Exception as e:
        logger.error(f"Malformed JSON payload: {e}")
        raise HTTPException(status_code=400, detail="Invalid payload model")

    logger.info(f"Processing webhook containing {len(payload.alerts)} alerts.")

    for alert in payload.alerts:
        if alert.status != "firing":
            logger.info(f"Ignoring resolved or inactive alert:
{alert.labels.get('alertname')}")
            continue

        alert_name = alert.labels.get("alertname", "UnknownAlert")
        severity = alert.labels.get("severity", "warning")
        pod_name = alert.labels.get("pod")
        namespace = alert.labels.get("namespace", "default")

        logger.info(f"ALERT FIRING: {alert_name} | Pod: {pod_name} | Namespace: {namespace}")

        # Automated Diagnostic and Self-Healing Logic Tree
```

```
        if alert_name == "HighFileDescriptorLeak":
            if pod_name:
                # Step 1: Capture Forensics
                logs = await IncidentRemediator.capture_pod_logs(namespace, pod_name)

                # Step 2: Trigger Self-Healing Mitigation
                remediation_success = await
IncidentRemediator.recycle_kubernetes_pod(namespace, pod_name)
                action_desc = f"Recycled Pod {pod_name} due to critical FD Leak" if
remediation_success else "Failed Pod Recycle"

                # Step 3: Dispatch RCA Report
                await IncidentRemediator.dispatch_slack_rca(
                    title=f"File Descriptor Leak on {pod_name}",
                    severity=severity,
                    details={"pod": pod_name, "namespace": namespace, "action": action_desc},
                    logs=logs
                )

        elif alert_name == "EventLoopBlockedCritical":
            if pod_name:
                logs = await IncidentRemediator.capture_pod_logs(namespace, pod_name)
                # Soft action: Pod restart
                remediation_success = await
IncidentRemediator.recycle_kubernetes_pod(namespace, pod_name)

                await IncidentRemediator.dispatch_slack_rca(
                    title=f"Async Event Loop Starvation on {pod_name}",
                    severity=severity,
                    details={"pod": pod_name, "namespace": namespace, "action": "Terminated
hung process container"},
                    logs=logs
                )

    return {"status": "processing_complete", "timestamp": time.time()}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080, log_level="info")
```

---

7. Deep Dive CLI Diagnostic Tools & Python Introspection Equivalents

SRE professionals must understand how command-line diagnostic utilities translate to lower-level Linux interfaces and standard Python libraries:

```
                          SYSTEM DIAGNOSTIC LAYERS
????????????????????????????????????????????????????????????????????????????????
? CLI Tools:  `lsof`       `gdb`/`py-spy`     `top`/`vmstat`   `tcpdump`      ?
????????????????????????????????????????????????????????????????????????????????
? Python:     `psutil`     `sys._current_`    `/proc` parser   `socket`       ?
????????????????????????????????????????????????????????????????????????????????
? Linux Kernel: VFS /proc    ptrace()         Cgroups / sysfs  Netfilter/skb  ?
????????????????????????????????????????????????????????????????????????????????
```

1. File Descriptor & Socket Inspection

- **CLI Command**: `lsof -p <PID>` or `ls -la /proc/<PID>/fd`
- **Deep Explanation**: Inspects open File Descriptors mapped in the kernel's Virtual File System (VFS). Elevated counts signal leak conditions (e.g., unclosed TCP sockets or DB connections).
- **Python Equivalent**:

```python
import os, sys, psutil

def inspect_process_fds(pid: int):
    proc = psutil.Process(pid)
    open_files = proc.open_files()
    connections = proc.net_connections()
    print(f"PID {pid} | Total FDs: {proc.num_fds()}")
    print(f"Open Files: {len(open_files)} | Open Network Sockets: {len(connections)}")
    for conn in connections:
        print(f"  Local: {conn.laddr} -> Remote: {conn.raddr} | State: {conn.status}")
```

2. Live Process Stack Tracing (C/Python Frame Extraction)

- **CLI Command**: `py-spy dump --pid <PID>` or `gdb -p <PID>`
- **Deep Explanation**: Uses the `ptrace(2)` system call to read CPython interpreter state out-of-process without pausing the targeted thread.
- **Python In-Process Equivalent**:

```python
import sys, traceback

def dump_all_thread_stack_traces():
    """Outputs current stack frames of all execution threads."""
    print("\n--- LIVE THREAD STACK DUMP ---")
    for thread_id, frame in sys._current_frames().items():
        print(f"\nThread ID: {thread_id}")
        traceback.print_stack(frame)
    print("--- END STACK DUMP ---\n")
```

3. Linux Proc Filesystem Direct Memory Diagnostics

- **CLI Command**: `cat /proc/<PID>/smaps_rollup` or `vmstat 1`
- **Deep Explanation**: Returns accurate kernel accounting of proportional set size (PSS), resident set size (RSS), and swap usage, avoiding subshell overhead.
- **Python Equivalent**:

```
def parse_proc_meminfo(pid: int) -> dict:
    """Directly parses /proc/[pid]/smaps_rollup without shelling out."""
    metrics = {}
    with open(f"/proc/{pid}/smaps_rollup", "r") as f:
        for line in f:
            parts = line.split()
            if len(parts) >= 2:
                key = parts[0].rstrip(":")
                val = int(parts[1])  # KiB
                metrics[key] = val
    return metrics # Returns dict with Rss, Pss, Shared_Clean, Shared_Dirty, etc.
```

---

8. Production Configuration Examples

Alertmanager Configuration (`alertmanager.yml`)

Configured to forward high-severity FD and Event Loop alerts to the Python Remediation Service with HMAC security headers:

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'namespace', 'pod']
  group_wait: 10s
  group_interval: 30s
  repeat_interval: 2h
  receiver: 'python-remediation-engine'
  routes:
    - match:
        severity: critical
      receiver: 'python-remediation-engine'
      continue: true

receivers:
- name: 'python-remediation-engine'
  webhook_configs:
  - url: 'http://sre-remediation-engine.monitoring.svc.cluster.local:8080/api/v1/remediate'
    send_resolved: false
```

```
    http_config:
      headers:
        X-Signature-256: "sha256=c89b7d2e3f4a1b0c9d8e7f6a5b4c3d2e"
      bearer_token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.SRE_SECURE_TOKEN"
```

Prometheus Alerting Rules (`prometheus_rules.yml`)

```
groups:
- name: sre_python_automation_rules
  rules:

  - alert: HighFileDescriptorLeak
    expr: (sre_process_open_file_descriptors / sre_process_max_file_descriptors) * 100 > 80
    for: 1m
    labels:
      severity: critical
      tier: infrastructure
    annotations:
      summary: "Process file descriptor utilization exceeded 80%"
      description: "Pod {{ $labels.pod }} in namespace {{ $labels.namespace }} is leaking
File Descriptors (Current: {{ $value }}%). Imminent port/file exhaustion."

  - alert: EventLoopBlockedCritical
    expr: histogram_quantile(0.99, sum(rate(sre_asyncio_event_loop_lag_seconds_bucket[5m]))
by (le, pod, namespace)) > 0.5
    for: 30s
    labels:
      severity: critical
      tier: application
    annotations:
      summary: "Asyncio Event Loop blocked for over 500ms"
      description: "Async thread starvation detected on Pod {{ $labels.pod }}. Event loop lag
exceeds 500ms at p99."
```

Deployment Configuration for Python Diagnostic Daemon (`daemon.yaml`)

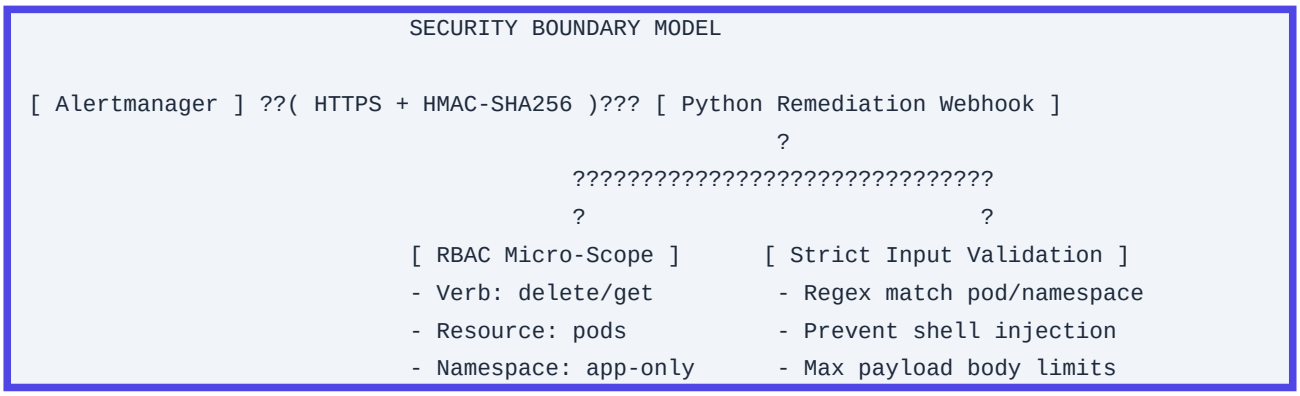
```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: sre-python-diagnostic-exporter
  namespace: monitoring
  labels:
    app.kubernetes.io/name: sre-diagnostic-exporter
spec:
  selector:
    matchLabels:
```

```
      app: sre-diagnostic-exporter
  template:
    metadata:
      labels:
        app: sre-diagnostic-exporter
    spec:
      hostPID: true # Requires PID namespace access to inspect target processes on node
      serviceAccountName: sre-exporter-sa
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        fsGroup: 65534
        seccompProfile:
          type: RuntimeDefault
      containers:
      - name: exporter
        image: internal-registry.enterprise.io/sre/python-exporter:v1.4.2
        imagePullPolicy: IfNotPresent
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities:
            drop:
            - ALL
            add:
            - SYS_PTRACE # Permitted for reading process metadata safely
        resources:
          limits:
            cpu: 100m
            memory: 128Mi
          requests:
            cpu: 20m
            memory: 64Mi
        ports:
        - containerPort: 9108
          name: metrics
        readinessProbe:
          httpGet:
            path: /metrics
            port: 9108
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /metrics
            port: 9108
          initialDelaySeconds: 15
          periodSeconds: 20
```

---

9. Security Considerations & Hardening Best Practices

Deploying automated remediation daemons that possess control-plane capabilities (e.g., terminating pods, adjusting cloud resources) introduces significant security attack surfaces.



1. Cryptographic Webhook Authentication

Never expose webhook endpoints without payload verification. Use SHA256 HMAC (Hash-based Message Authentication Code) validation.

```
# In FastAPI request handlers:
expected_mac = hmac.new(SECRET_KEY, request_body, hashlib.sha256).hexdigest()
if not hmac.compare_digest(request_header_mac, expected_mac):
    raise PermissionDenied("Cryptographic Signature Mismatch")
```

2. Micro-Scoped Kubernetes RBAC Authorization

The ServiceAccount bound to the Python remediation container must follow the Principle of Least Privilege. Never assign `cluster-admin`.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: sre-remediation-role
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "delete"] # Strictly limit actions to specified operations
```

3. Defense Against Command Injection

- **Never shell out using `os.system()` or `subprocess.Popen(shell=True)`** with variables extracted from

metric labels or alert bodies.

- Use official native SDKs (`kubernetes`, `boto3`, `google-cloud-storage`) which handle request serialization safely.

---

10. Observability & Monitoring of SRE Automation Components

Meta-observability (monitoring the tools that monitor the system) is critical. If the Python Remediation Engine crashes silently, critical outages go unhandled.

Key Metrics to Monitor on Exporters and Remediation Daemons

| Metric Name | Type | Critical Alert Threshold | SLA Impact |

| :--- | :--- | :--- | :--- |

| `sre_remediation_execution_duration_seconds` | Histogram | p99 > 15.0 seconds | Slow MTTR |

| `sre_remediation_failures_total` | Counter | Rate > 0 over 5m | Failed self-healing action |

| `sre_exporter_collection_errors_total` | Counter | Rate > 0.1/sec | Observability telemetry loss |

| `sre_asyncio_event_loop_lag_seconds` | Histogram | p95 > 0.1 seconds | Engine thread contention |

Structured Logging Requirement

All SRE scripts must produce single-line JSON output to stdout to support log aggregators (e.g., Vector, FluentBit, Datadog):

```
{
  "timestamp": "2023-10-24T14:32:01.002931Z",
  "level": "WARN",
  "correlation_id": "req-8f921a-alertmanager",
  "event": "POD_RECYCLE_INITIATED",
  "target_pod": "payments-api-7d9b5c-x29f",
  "namespace": "production",
  "reason": "HighFileDescriptorLeak",
  "duration_ms": 142.8
}
```

---

11. Common Troubleshooting Scenarios with Step-by-Step RCA

Scenario A: Asyncio Event Loop Deadlock (Thread Starvation)

- **Symptom**: An asynchronous Python API service stops accepting connections. CPU usage sits at 100% on a single core, while HTTP health checks time out.
- **Diagnostic Execution via Python**:

1. Trigger dynamic process stack capture via low-level signal interception:

```
import faulthandler
import signal

# Call this during service bootstrap
def enable_async_diagnostics():
    # Sending SIGUSR1 to process dumps all C Python stacks to stderr
    faulthandler.register(signal.SIGUSR1, all_threads=True)
```

2. Send signal from CLI: `kill -s SIGUSR1 <PID>`

3. Root Cause Identification: Stacks show an async function invoking a synchronous blocking DB call (e.g., `requests.get()` or `time.sleep()`) directly inside the event loop instead of offloading to `asyncio.to_thread()`.

Scenario B: Ephemeral File Descriptor & Socket Leakage

- **Symptom**: Pod fails with `OSError: [Errno 24] Too many open files`.
- **Diagnostic Execution via Python**:

1. Execute programmatic FD analysis:

```
import os, psutil

def trace_leaking_descriptors():
    proc = psutil.Process(os.getpid())
    fd_map = {}
    for fd in proc.open_files():
        ext = os.path.splitext(fd.path)[1]
        fd_map[ext] = fd_map.get(ext, 0) + 1

    # Check socket descriptors
    sockets = [c for c in proc.net_connections()]
    print(f"File breakdown: {fd_map}")
    print(f"Active sockets count: {len(sockets)}")
```

2. Root Cause Identification: HTTP client sessions (`aiohttp.ClientSession()`) were instantiated inside request handlers rather than created once as application-level singletons, leaking unclosed TCP sockets in `TIME_WAIT` status.

---

12. Common Mistakes and How to Avoid Them

1. Blocking Scrape Collectors

- **Anti-Pattern**: Performing synchronous HTTP requests or slow system disk scans inside Prometheus collector functions, causing metric scrape timeouts (HTTP 504).
- **Prevention**: Offload heavy synchronous calls to `loop.run_in_executor()` or collect metrics asynchronously in background loops, caching results in memory for the exporter scrapers.

2. High Cardinality Metric Explosion

- **Anti-Pattern**: Dynamic values (such as IP addresses, User IDs, or raw Error Strings) set as Prometheus labels.

```
# BAD: High Cardinality! Destroys Prometheus TSDB performance.
ERRORS.labels(user_id=user.id, raw_url=request.url).inc()
```

- **Prevention**: Sanitize and group dynamic values into bounded label sets.

```
# GOOD: Low Cardinality.
ERRORS.labels(endpoint_group="/api/v1/checkout", status_code=500).inc()
```

3. Cascading Self-Healing Remediation Loops

- **Anti-Pattern**: An automated engine continuously deletes a failing pod. The pod reboots, immediately refires the alert, and enters an infinite restart loop.
- **Prevention**: Implement persistent state locks (e.g., Redis or in-memory LRU backpressure) limiting remediation actions to a maximum threshold (e.g., *maximum 2 restarts per pod per 30 minutes*).

---

13. Enterprise Performance Tuning and Optimization

For Python diagnostic scripts executing on large-scale infrastructure (thousands of nodes/pods):

1. Memory Optimization via `__slots__`

When parsing large process maps or system states in loop iterations, eliminate default instance `__dict__` overhead by declaring `__slots__`.

```
class MetricPoint:
    __slots__ = ['name', 'value', 'timestamp', 'labels']
    def __init__(self, name: str, value: float, timestamp: float, labels: dict):
        self.name = name
        self.value = value
        self.timestamp = timestamp
        self.labels = labels
```

2. HTTP Connection Pooling for Alerts & Webhooks

Never instantiate fresh HTTP clients per request payload inside high-throughput remediation loops. Reuse pooled instances:

```
# Application-wide global connection pool engine
HTTP_ASYNC_CLIENT = httpx.AsyncClient(
    limits=httpx.Limits(max_keepalive_connections=20, max_connections=100),
    timeout=httpx.Timeout(connect=2.0, read=5.0, write=5.0, pool=10.0)
)
```

---

14. Advanced Concepts: Real-time Kernel Tracing with Python eBPF

For ultra-low-overhead system diagnostics, enterprise SRE teams bypass `/proc` entirely by loading kernel eBPF bytecode using Python interface bindings (`bcc`).

```
#!/usr/bin/env python3
"""
Python eBPF Listener: Traces Kernel Level TCP Connection Restats/Drops
Requires: bpfcc (BPF Compiler Collection bindings) and Root/CAP_SYS_ADMIN.
"""
import sys

try:
    from bcc import BPF
except ImportError:
    print("eBPF (bcc) library missing. Executing in fallback context.")
    sys.exit(0)

# Inline C Kernel Code for eBPF Instrumenting
bpf_text = """
#include <uapi/linux/ptrace.h>
#include <net/sock.h>

BPF_PERF_OUTPUT(tcp_drop_events);

struct drop_evt_t {
    u32 pid;
    u64 ip_version;
};

int trace_tcp_drop(struct pt_regs *ctx, struct sock *sk) {
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    struct drop_evt_t evt = {};
    evt.pid = pid;
```

```
    tcp_drop_events.perf_submit(ctx, &evt, sizeof(evt));
    return 0;
}
"""

def start_kernel_tracing():
    # Compile and Inject eBPF C program into Kernel space
    b = BPF(text=bpf_text)
    b.attach_kprobe(event="tcp_drop", fn_name="trace_tcp_drop")
    print("Successfully injected eBPF kernel probes. Tracing TCP drops...")

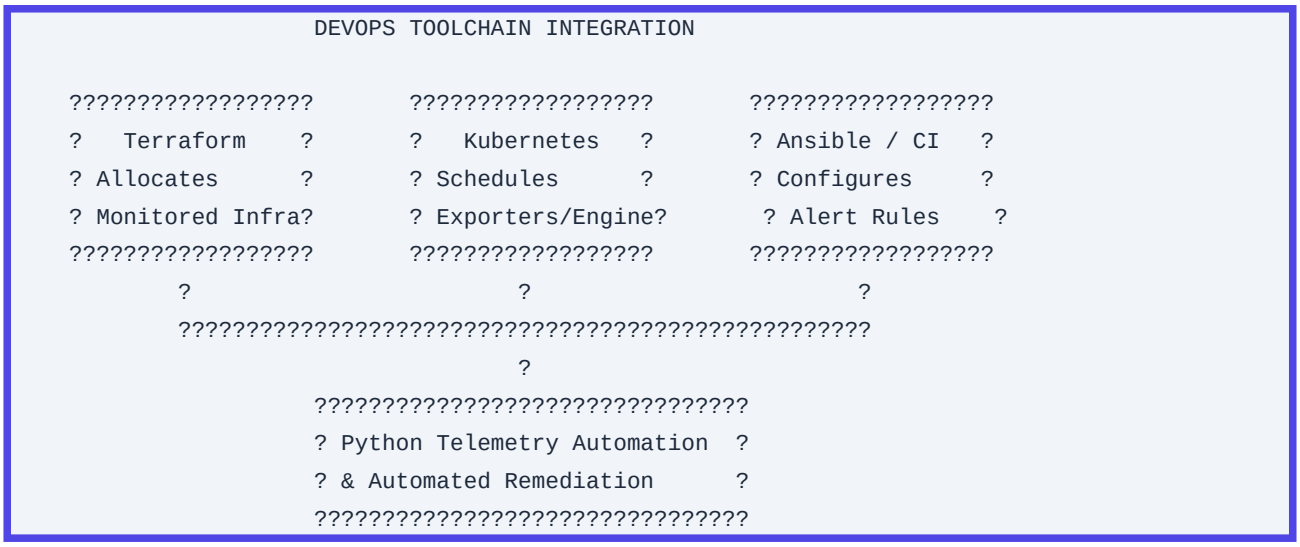
    def print_drop_event(cpu, data, size):
        event = b["tcp_drop_events"].event(data)
        print(f"[KERNEL ALERT] TCP Drop Event Detected for PID: {event.pid}")

    b["tcp_drop_events"].open_perf_buffer(print_drop_event)
    while True:
        try:
            b.perf_buffer_poll()
        except KeyboardInterrupt:
            sys.exit(0)

if __name__ == "__main__":
    start_kernel_tracing()
```

---

15. Integration with Other DevOps Tools



1. Kubernetes Integration: Python diagnostic containers leverage native client libraries (`kubernetes`) to interact directly with cluster controllers, managing runtime mutations (scaling up, evicting pods, updating annotations).

2. Terraform Integration: Provision security credentials, Webhook URL secrets (AWS Secrets Manager / Vault), and IAM roles consumed by Python remediation engines automatically using Terraform resources.

3. CI/CD Integration: Verify Prometheus alerting rule validity and custom metric structures during build pipelines by invoking test suites via Python `pytest` plugins before rolling out deployments.

---

16. Comparison Matrix: Python SRE Tooling vs. Alternatives

| Feature / Metric | Custom Python SRE Engine | Node Exporter / Vector | StackStorm / Robusta |

| :--- | :--- | :--- | :--- |

| Primary Focus | Deep Application Diagnostics & Remediation | System Metric Gathering / Log Forwarding | Event-Driven Workflow Automation |

| Customizability | Maximum (Full Python standard library & SDK access) | Low (Static predefined metrics) | High (Python/YAML based actions) |

| Resource Footprint | Extremely Low (~30MB RAM, <1% CPU) | Low (~15MB RAM, C++/Rust performance) | Medium to High (Heavy multi-service engine) |

| In-Process Introspection | Yes (`sys._current_frames`, `tracemalloc`) | No (External monitoring only) | Partial (Relies on external probes) |

| Latency / Response Time | Sub-second execution | N/A (Telemetry collector only) | 2-5 seconds overhead |

| Setup Complexity | Low (Single standard Python service container) | Minimal (Precompiled binary/image) | High (Requires dedicated DB/Message Queue) |

---

17. Visual SRE Diagnostic Cheat Sheet

| Python Tool / Module | Target Primitive | Standard Enterprise Snippet / Syntax | SRE Diagnostic Application |

| :--- | :--- | :--- | :--- |

| `psutil.Process().num_fds()` | Open File Descriptors | `psutil.Process(pid).num_fds()` | Ephemeral port/file leak identification |

| `psutil.Process().memory_info()` | RSS / VMS Memory | `psutil.Process(pid).memory_info().rss` | Native OOM-Killed warning detection |

| `sys._current_frames()` | CPython Thread Frames | `sys._current_frames()` | Extraction of live application stack frame dumps |

| `prometheus_client` | Telemetry Metrics | `Gauge('metric_name', 'help', registry=r)` | Exposing custom domain metric endpoints |

| `faulthandler` | C Signal Handler | `faulthandler.register(signal.SIGUSR1)` | Stack tracing unresponsive

C-extensions |

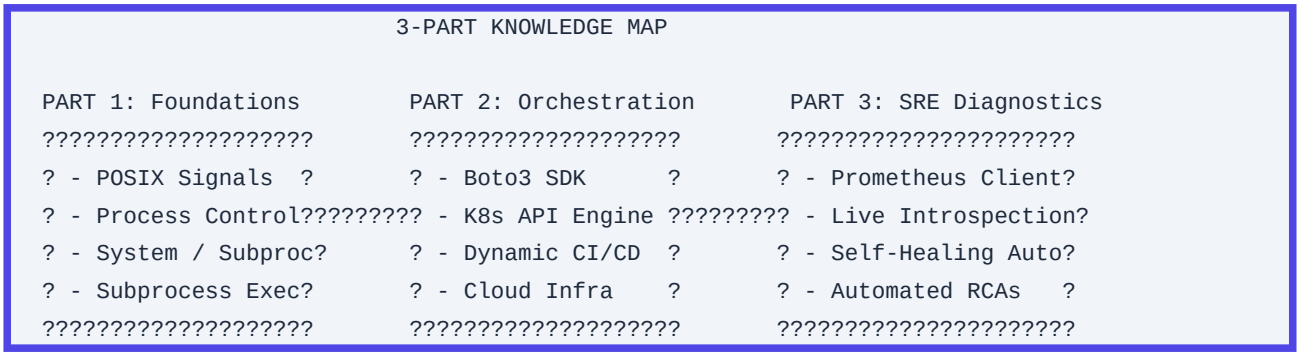
| `asyncio.Task.all_tasks()` | Async Execution Graph | `asyncio.all_tasks(loop=loop)` | Asyncio event loop starvation debugging |

| `tracemalloc` | Heap Allocation | `tracemalloc.take_snapshot().statistics('lineno')` | Memory leak code attribution |

---

18. Comprehensive Learning Summary (Parts 1 - 3)

Across this 3-part series, we have established the blueprint for leveraging Python as a core engineering standard within Cloud and SRE disciplines:



- **Part 1**: Mastered the core foundation: POSIX file systems, system calls, robust subshell execution using `subprocess`, signal handling, and clean CLI creation.
- **Part 2**: Scaled execution to Cloud Orchestration: Direct programmatic manipulation of AWS/Cloud services (`boto3`), native Kubernetes API custom controllers, dynamic pipeline generation, and infrastructure state validation.
- **Part 3**: Advanced to SRE Excellence: Deep-system runtime diagnostics, custom metric instrumentation using `prometheus_client`, non-blocking alert receivers, and automated incident response runbooks with instant forensic capture.

Adopting these programmatic patterns transitions an engineering team away from reactive manual operations ("toil") toward building deterministic, self-healing, highly available enterprise systems.

Q41. How do you diagnose, isolate, and programmatically mitigate memory leaks and RSS bloat in long-running Python Kubernetes workloads?

Detailed Answer:

Memory management in Python relies on reference counting combined with a generational garbage collector (GC) to resolve cyclic dependencies. In production Kubernetes workloads, Python applications frequently suffer from dynamic RSS (Resident Set Size) bloat due to three main causes:

1. Uncollected Circular References: Objects referencing each other that bypass immediate reference count drops

and reside in Generation 2 GC space.

2. C-Extension Memory Leaks: Native libraries (`C` bindings like `cffi`, `numpy`, `psycopg2`) allocating memory via raw `malloc()` outside Python's `pymalloc` allocator, rendering standard Python GC power loss.

3. Memory Fragmentation: `pymalloc` handles small objects (≤ 512 bytes) by allocating 256 KB Arenas divided into 4 KB Pools. When objects within a pool are freed non-contiguously, Python cannot yield the underlying arena back to the OS `glibc` `malloc` arena, leading to high Linux kernel RSS while Python's internal heap reports low usage.

To systematically diagnose memory leaks in production without causing performance degradation, SREs employ `tracemalloc` for allocation tracking, `gc` inspection for cycle debugging, and dynamic heap dump generation using libraries like `objgraph` or `guppy3`. When RSS approaches pod limits (triggering `OOMKilled` via kernel cgroup enforcement), a diagnostic endpoint or signal handler should execute snapshot comparisons between two runtime temporal states (T_0 and T_1).

Mitigation involves overriding `pymalloc` behaviors using environment variables like `PYTHONMALLOC=malloc` (forcing allocation to system `glibc` where memory allocators like `jemalloc` or `tcmalloc` reduce fragmentation via aggressive `MADV_DONTNEED` profiling), manually triggering `gc.collect()` at idle intervals, and explicitly invoking `malloc_trim(0)` from `ctypes` to force the `glibc` allocator to return freed top-heap memory to the OS kernel.

Production Scenario / Practical Example:

A FastAPI microservice running inside a Kubernetes pod exhibits a linear memory leak, consuming 15 MB per hour until hit by cgroup OOM termination (exit code 137). Below is a production-grade diagnostic and auto-mitigation module embedded in the service. It exposes a memory profiling endpoint and dynamically executes `glibc` trim operations when RSS exceeds critical thresholds.

```
import os
import gc
import ctypes
import tracemalloc
import logging
import psutil
from fastapi import FastAPI, Response, status

app = FastAPI()
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("SRE-Memory-Diagnostic")

# Start tracing memory allocations at startup (keep frame limit low for minimal overhead)
tracemalloc.start(10)
_snapshot_baseline = tracemalloc.take_snapshot()

def force_glibc_malloc_trim() -> int:
    """Invokes malloc_trim via ctypes to release glibc fragmented memory back to OS
kernel."""
    try:
```

```
        libc = ctypes.CDLL("libc.so.6")
        # malloc_trim returns 1 if memory was actually returned to the system
        result = libc.malloc_trim(0)
        logger.info(f"malloc_trim(0) executed. Result: {result}")
        return result
    except Exception as e:
        logger.error(f"Failed to execute malloc_trim: {e}")
        return 0

@app.get("/metrics/memory/leak-analysis")
def analyze_memory_leak():
    """Diagnostic endpoint comparing current memory snapshot against baseline."""
    global _snapshot_baseline
    current_snapshot = tracemalloc.take_snapshot()
    top_stats = current_snapshot.compare_to(_snapshot_baseline, 'lineno')

    diff_report = []
    for stat in top_stats[:10]: # Top 10 memory growth origins
        diff_report.append({
            "file": str(stat.traceback),
            "size_diff_kb": stat.size_diff / 1024,
            "count_diff": stat.count_diff
        })

    process = psutil.Process(os.getpid())
    rss_mb = process.memory_info().rss / (1024 * 1024)

    return {
        "pid": process.pid,
        "rss_mb": rss_mb,
        "gc_counts": gc.get_count(),
        "gc_objects_count": len(gc.get_objects()),
        "top_growth_allocations": diff_report
    }

@app.post("/sys/rebalance-heap", status_code=status.HTTP_200_OK)
def trigger_heap_rebalance(response: Response):
    """Emergency remediation endpoint invoked by SRE runbooks/k8s probes."""
    logger.warning("Emergency heap rebalance triggered. Running full GC cycle...")

    # Uncollectable garbage collection
    gc.collect(generation=2)

    # Release memory from C-allocator back to OS kernel
    trimmed = force_glibc_malloc_trim()

    process = psutil.Process(os.getpid())
    post_rss_mb = process.memory_info().rss / (1024 * 1024)
```

```
    return {
        "status": "success",
        "malloc_trim_success": bool(trimmed),
        "post_rebalance_rss_mb": post_rss_mb
    }
```

Q42. How do you design and implement high-performance custom Prometheus Exporters in Python while mitigating high-cardinality risks and memory growth?

Detailed Answer:

When standard exporters (e.g., `node_exporter`, `kube-state-metrics`) do not expose specialized domain or infrastructure metrics, SREs build custom Prometheus Exporters. Writing an enterprise-grade exporter in Python requires deep understanding of the `prometheus_client` library's underlying mechanics, scrape request lifecycle, thread safety, and label cardinalities.

High cardinality occurs when metric labels dynamically accept high-entropy values (e.g., User IDs, Session IDs, GUIDs, IPv6 addresses, dynamic URIs). In Prometheus, every unique combination of key-value pair labels constructs a distinct, immutable time-series in the TSDB. If an exporter generates 100,000 unique label combinations, the memory usage of both the exporter and the scraping Prometheus server will surge exponentially, causing Prometheus scrape timeouts or OOM crashes.

To prevent cardinality explosion:

1. Quantize/Normalize Label Values: Strip dynamic components from labels (e.g., replace `/api/v1/user/1029384` with `/api/v1/user/:id`).

2. Use Custom Collectors over Static Metrics: Instead of updating static global `Gauge` or `Counter` instances continuously throughout application runtime (which stores metric states indefinitely in memory), implement custom collectors extending `prometheus_client.registry.Collector`. The custom collector overrides the `collect()` method to compute metrics lazily and yield them on-demand *only* when Prometheus executes an HTTP GET scrape.

3. Thread Safety and Performance: Use atomic operations or dynamic lock abstractions when mutating shared internal structures during calculation.

Production Scenario / Practical Example:

The following script is a production custom exporter that queries an internal Redis queue cluster, dynamically normalizes worker task identifiers to control cardinality, and implements a custom lazy `Collector` pattern to guarantee zero residual metric memory growth between scrapes.

```
import time
import re
import logging
from typing import Iterable
```

```
from prometheus_client import start_http_server, Metric
from prometheus_client.core import CounterMetricFamily, GaugeMetricFamily, REGISTRY
import redis

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")

class RedisQueueCustomCollector:
    def __init__(self, redis_host: str, redis_port: int):
        self.redis_client = redis.Redis(host=redis_host, port=redis_port, socket_timeout=2.0)
        # Regex pattern to match UUIDs and numeric IDs to prevent high-cardinality
        self.dynamic_id_pattern = re.compile(r'/[0-9a-fA-F-]{36}|/\d+')

    def _normalize_queue_name(self, raw_name: str) -> str:
        """Strips unique IDs from queue names to keep label cardinality low."""
        return self.dynamic_id_pattern.sub('/:id', raw_name)

    def collect(self) -> Iterable[Metric]:
        """Invoked synchronously by prometheus_client whenever /metrics is scraped."""
        gauge_queue_depth = GaugeMetricFamily(
            'app_queue_depth_messages',
            'Current number of pending messages in the worker queues',
            labels=['queue_name', 'priority']
        )
        counter_scrape_errors = CounterMetricFamily(
            'app_exporter_scrape_errors_total',
            'Total scrape failure count from backend Redis'
        )

        try:
            # Fetch all queue keys from Redis safely using SCAN (never use KEYS in prod)
            cursor = 0
            queue_keys = []
            while True:
                cursor, keys = self.redis_client.scan(cursor=cursor, match="queue:*",
count=100)
                queue_keys.extend(keys)
                if cursor == 0:
                    break

            for key_bytes in queue_keys:
                key_str = key_bytes.decode('utf-8')
                # Extract components and normalize to control cardinality
                parts = key_str.split(':')
                raw_queue_name = parts[1] if len(parts) > 1 else 'unknown'
                priority = parts[2] if len(parts) > 2 else 'normal'

                normalized_name = self._normalize_queue_name(raw_queue_name)
                length = self.redis_client.llen(key_str)
```

```
                # Yield metric sample directly into collector context
                gauge_queue_depth.add_metric([normalized_name, priority], float(length))

            yield gauge_queue_depth

        except Exception as e:
            logging.error(f"Failed to scrape Redis queue metrics: {e}")
            counter_scrape_errors.add_metric([], 1.0)
            yield counter_scrape_errors

if __name__ == '__main__':
    EXPORTER_PORT = 9108
    # Unregister default collectors (ProcessCollector, GCCollector) if pure exporter
isolation is required
    REGISTRY.register(RedisQueueCustomCollector(redis_host='localhost', redis_port=6379))

    start_http_server(EXPORTER_PORT)
    logging.info(f"Custom Prometheus Exporter successfully started on port {EXPORTER_PORT}")

    # Keep main thread alive
    while True:
        time.sleep(3600)
```

Q43. How do you automate Kubernetes incident Root Cause Analysis (RCA) and diagnostic data collection using Python during Pod CrashLoopBackOff/OOMKilled events?

Detailed Answer:

When Kubernetes Pods crash repeatedly (`CrashLoopBackOff`) or get killed by the Linux OOM killer (`OOMKilled`), standard logging pipelines (e.g., Fluentbit/Loki) often lose the final stack traces due to buffer flushes being terminated prematurely by `SIGKILL`. Automated incident RCA scripts built with the official Python `kubernetes` client SDK allow SREs to monitor real-time cluster events via the Watch API, extract crucial diagnostic signals, parse previous container execution termination states, and post enriched RCA payloads directly to incident response channels (e.g., Slack, PagerDuty, or Jira).

Key signals extracted during RCA automation:

1. Termination Reasons and Exit Codes: Exit code `137` indicates `SIGKILL` (frequently OOM or failed liveness probe), `139` indicates `Segmentation Fault`, and `1` indicates an unhandled application exception.

2. Previous Container Logs: Querying `read_namespaced_pod_log(..., previous=True)` to retrieve the logs directly prior to the restart.

3. K8s Event Stack: Correlating `Warning` level events (e.g., `FailedScheduling`, `Unhealthy`, `FailedMount`) occurring around the exact `lastTimestamp` of the failure.

Production Scenario / Practical Example:

The following script acts as an automated incident RCA engine daemon inside a Kubernetes cluster. It watches for container failures in real-time, extracts the previous termination reason, fetches the last 50 lines of logs prior to crash, isolates the triggering error, and constructs a structured markdown RCA report.

```
import os
import json
import logging
from kubernetes import client, config, watch
import requests

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
logger = logging.getLogger("RCA-Engine")

# Initialize Kubernetes API client (In-Cluster or KubeConfig)
try:
    config.load_incluster_config()
except config.ConfigException:
    config.load_kube_config()

v1_api = client.CoreV1Api()

SLACK_WEBHOOK_URL = os.getenv("SLACK_WEBHOOK_URL",
"https://hooks.slack.com/services/mock/token")

def dispatch_rca_report(pod_name: str, namespace: str, container_name: str, exit_code: int,
reason: str, logs: str):
    """Formats and dispatches the RCA report to Slack."""
    payload = {
        "text": f"? *Automated RCA Report: Pod Failure Detected in {namespace}*",
        "attachments": [
            {
                "color": "#FF0000",
                "fields": [
                    {"title": "Pod Name", "value": pod_name, "short": True},
                    {"title": "Namespace", "value": namespace, "short": True},
                    {"title": "Container Name", "value": container_name, "short": True},
                    {"title": "Exit Code / Reason", "value": f"`{exit_code}` / `{reason}`",
"short": True},
                    {"title": "Crash Log Snippet (Previous Instance)", "value":
f"```\n{logs[-1500:]}\n```", "short": False}
                ]
            }
        ]
    }
    try:
        res = requests.post(SLACK_WEBHOOK_URL, json=payload, timeout=5.0)
        logger.info(f"RCA report dispatched to Slack. HTTP status: {res.status_code}")
```

```
    except Exception as e:
        logger.error(f"Failed to deliver RCA report: {e}")

def monitor_cluster_failures():
    """Watches Pod status changes across namespaces for restart anomalies."""
    w = watch.Watch()
    logger.info("RCA Engine Active. Watching cluster Pod lifecycle events...")

    for event in w.stream(v1_api.list_pod_for_all_namespaces):
        pod = event['object']
        pod_name = pod.metadata.name
        namespace = pod.metadata.namespace

        if not pod.status.container_statuses:
            continue

        for cs in pod.status.container_statuses:
            # Inspect last state to detect recent crashes
            terminated = cs.last_state.terminated
            if terminated and cs.restart_count > 0:
                # Check if this state update corresponds to a fresh container restart
                exit_code = terminated.exit_code
                reason = terminated.reason or "Unknown"
                container_name = cs.name

                logger.warning(f"Detected restart on {namespace}/{pod_name}
({container_name}). Code: {exit_code}, Reason: {reason}")

                # Fetch last lines prior to termination
                try:
                    prev_logs = v1_api.read_namespaced_pod_log(
                        name=pod_name,
                        namespace=namespace,
                        container=container_name,
                        previous=True,
                        tail_lines=50
                    )
                except Exception as ex:
                    prev_logs = f"Unable to fetch previous logs: {str(ex)}"

                dispatch_rca_report(pod_name, namespace, container_name, exit_code, reason,
prev_logs)

if __name__ == "__main__":
    monitor_cluster_failures()
```

Q44. How do you construct an asynchronous, high-throughput log parser and anomaly detector using Python's `asyncio` and `ProcessPoolExecutor`?

Detailed Answer:

High-throughput log analysis in SRE tooling involves combining CPU-bound tasks (Regex compilation, pattern extraction, sliding-window math calculation) with I/O-bound tasks (reading network streams, tailing log files from disks, dispatching HTTP webhook alerts).

Because the Python Global Interpreter Lock (GIL) prevents concurrent execution of native Python bytecode across multi-core systems, executing heavy regex operations sequentially inside an `asyncio` event loop degrades performance, blocks the loop tick, and leads to dropped network socket packets.

To construct a scalable SRE diagnostic log pipeline capable of processing 50,000+ log lines per second:

1. Asynchronous I/O Pipeline (`asyncio`): Use non-blocking I/O (`aiofiles` or `asyncio.StreamReader`) to stream log inputs into memory buffers without stalling concurrent operations.

2. CPU Offloading (`concurrent.futures.ProcessPoolExecutor`): Delegate heavy parsing operations and statistical anomaly analysis across separate OS process workers via `loop.run_in_executor()`. This bypasses the GIL completely.

3. Sliding Window Anomaly Detection: Implement a rolling multi-second time-window aggregate in memory (e.g., tracking the ratio of `ERROR`/`FATAL` logs vs total logs over a 60-second moving window) to detect metric anomalies prior to centralized ingestion pipelines.

Production Scenario / Practical Example:

Below is an enterprise diagnostic worker script reading from a high-volume log stream asynchronously, leveraging a multi-core process pool to parse regex patterns concurrently, and computing error-rate density anomalies across a rolling 10-second window.

```
import asyncio
import re
import time
from collections import deque
from concurrent.futures import ProcessPoolExecutor
import os

# Pure function that runs inside separate OS Process (GIL-bypassing)
def parse_and_classify_log(raw_line: str) -> dict:
    """CPU-bound task for regex parsing and severity identification."""
    # Common Log Format + Severity pattern matching
    pattern = r'(?P<ip>\S+) \S+ \S+ \[(?P<timestamp>.*?)\] "(?P<method>\S+) (?P<path>\S+)
\S+" (?P<status>\d{3}) (?P<bytes>\d+) severity=(?P<severity>\w+)'
    match = re.search(pattern, raw_line)

    if match:
        data = match.groupdict()
        return {
```

```
            "valid": True,
            "status": int(data["status"]),
            "severity": data["severity"].upper(),
            "path": data["path"]
        }
    else:
        # Fallback check for unformatted stack traces
        is_error = "ERROR" in raw_line or "EXCEPTION" in raw_line or "CRITICAL" in raw_line
        return {
            "valid": False,
            "status": 500 if is_error else 200,
            "severity": "ERROR" if is_error else "INFO",
            "path": "unknown"
        }

class AsyncLogAnomalyDetector:
    def __init__(self, error_threshold_ratio: float = 0.15, window_seconds: int = 10):
        self.error_threshold_ratio = error_threshold_ratio
        self.window_seconds = window_seconds
        self.sliding_window = deque() # Stores tuples: (timestamp, is_error: bool)
        self.executor = ProcessPoolExecutor(max_workers=os.cpu_count())

    def _evaluate_anomaly(self) -> bool:
        """Computes current error ratio in sliding window."""
        now = time.time()
        # Purge entries older than window limit
        while self.sliding_window and self.sliding_window[0][0] < (now -
self.window_seconds):
            self.sliding_window.popleft()

        if not self.sliding_window:
            return False

        total_logs = len(self.sliding_window)
        total_errors = sum(1 for _, is_error in self.sliding_window if is_error)
        error_ratio = total_errors / total_logs

        if error_ratio > self.error_threshold_ratio and total_logs > 20:
            print(f"? ANOMALY DETECTED! Sliding Window Error Rate: {error_ratio*100:.2f}%
({total_errors}/{total_logs} logs)")
            return True
        return False

    async def process_log_stream(self, stream_reader: asyncio.StreamReader):
        loop = asyncio.get_running_loop()
        print("Starting Asynchronous Stream Processing Pipeline...")

        while not stream_reader.at_eof():
```

```
            line_bytes = await stream_reader.readline()
            if not line_bytes:
                break

            line_str = line_bytes.decode('utf-8', errors='ignore')

            # Offload heavy parsing to ProcessPoolExecutor (GIL bypass)
            parsed_result = await loop.run_in_executor(
                self.executor, parse_and_classify_log, line_str
            )

            is_error = parsed_result["severity"] in ["ERROR", "CRITICAL", "FATAL"] or
parsed_result["status"] >= 500
            self.sliding_window.append((time.time(), is_error))

            self._evaluate_anomaly()

# Driver snippet demonstrating stream processing
async def main():
    detector = AsyncLogAnomalyDetector(error_threshold_ratio=0.2, window_seconds=5)

    # Simulate high-volume log input stream via Pipe
    read_fd, write_fd = os.pipe()
    reader = asyncio.StreamReader()
    protocol = asyncio.StreamReaderProtocol(reader)
    loop = asyncio.get_running_loop()
    await loop.connect_read_pipe(lambda: protocol, os.fdopen(read_fd, 'rb'))

    # Mock writer task generating raw log streams
    async def mock_log_generator():
        writer = os.fdopen(write_fd, 'wb', buffering=0)
        for i in range(100):
            if i % 3 == 0:
                log = f'10.0.0.1 - - [10/Oct/2023:13:55:36 +0000] "GET /api/pay HTTP/1.1" 500
128 severity=ERROR\n'
            else:
                log = f'10.0.0.1 - - [10/Oct/2023:13:55:36 +0000] "GET /api/health HTTP/1.1"
200 64 severity=INFO\n'
            writer.write(log.encode('utf-8'))
            await asyncio.sleep(0.01)
        writer.close()

    await asyncio.gather(
        detector.process_log_stream(reader),
        mock_log_generator()
    )

if __name__ == '__main__':
```

```
    asyncio.run(main())
```

Q45. How do you implement a robust, idempotent automated self-healing framework in Python triggered by Prometheus Alertmanager webhooks?

Detailed Answer:

Self-healing auto-remediation architectures use webhooks dispatched by Prometheus Alertmanager to execute corrective actions without human intervention. Building an enterprise-grade auto-remediation receiver in Python requires strict software patterns to ensure systems remain stable:

1. Idempotency: Alerts can be fired repeatedly by Alertmanager during flapping state intervals. The remediation execution engine must check the actual state of the infrastructure (e.g., via AWS API or K8s API) before executing an action, guaranteeing that running the action N times yields the exact same outcome as running it once.

2. Circuit Breaking and Flapping Protection: Auto-remediators must enforce stateful limits (e.g., maximum 3 execution attempts per resource per hour). If remediation fails continuously, the engine breaks the circuit and escalates to a PagerDuty incident to prevent infinite restart cascades.

3. Webhook Verification and Authentication: Validating incoming HMAC authorization signatures or tokens to prevent unauthorized infrastructure mutations.

Production Scenario / Practical Example:

An infrastructure alert `PostgreSQLConnectionPoolExhausted` triggers an Alertmanager webhook. The Python self-healing receiver validates the alert context, checks an internal local state database/cache to ensure it hasn't exceeded its flap threshold, verifies the target database state, terminates idle leaked connections safely, and logs the mitigation history.

```
from flask import Flask, request, jsonify
import time
import hmac
import hashlib
import psycopg2
import logging
import os

app = Flask(__name__)
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("AutoRemediation")

WEBHOOK_SECRET = os.getenv("ALERTMANAGER_SECRET", "super-secret-token")
DB_CONN_STRING = os.getenv("DATABASE_URL", "postgresql://user:pass@localhost:5432/prod_db")

# Circuit Breaker Tracker: Maps target resource to timestamp history
```

```
REMEDIATION_HISTORY = {} # Format: { "resource_id": [timestamp1, timestamp2] }
MAX_REMEDIATIONS_PER_HOUR = 3

def is_circuit_broken(resource_id: str) -> bool:
    """Verifies that auto-remediation hasn't exceeded safety invocation thresholds."""
    now = time.time()
    history = REMEDIATION_HISTORY.get(resource_id, [])
    # Filter timestamps within the last 3600 seconds
    valid_history = [t for t in history if now - t < 3600]
    REMEDIATION_HISTORY[resource_id] = valid_history

    return len(valid_history) >= MAX_REMEDIATIONS_PER_HOUR

def record_remediation_event(resource_id: str):
    REMEDIATION_HISTORY.setdefault(resource_id, []).append(time.time())

def terminate_idle_db_connections(db_name: str) -> int:
    """Idempotent remediation task: Terminates idle backends older than 5 minutes."""
    conn = psycopg2.connect(DB_CONN_STRING)
    conn.autocommit = True
    cursor = conn.cursor()

    # Query to kill idle connections without destroying active transactions
    query = """
    SELECT pg_terminate_backend(pid)
    FROM pg_stat_activity
    WHERE datname = %s
      AND state = 'idle'
      AND state_change < NOW() - INTERVAL '5 minutes'
      AND pid <> pg_backend_pid();
    """
    cursor.execute(query, (db_name,))
    terminated_count = cursor.rowcount
    cursor.close()
    conn.close()
    return terminated_count

@app.route("/remediate/alertmanager", methods=["POST"])
def handle_alertmanager_webhook():
    # 1. Authenticate Request
    token = request.headers.get("X-Alertmanager-Secret")
    if not token or not hmac.compare_digest(token, WEBHOOK_SECRET):
        return jsonify({"status": "unauthorized"}), 401

    payload = request.get_json()
    alerts = payload.get("alerts", [])

    results = []
```

```
    for alert in alerts:
        status = alert.get("status")
        labels = alert.get("labels", {})
        alert_name = labels.get("alertname")
        target_db = labels.get("database_name", "prod_db")

        # Ignore resolved notifications
        if status != "firing":
            continue

        if alert_name == "PostgreSQLConnectionPoolExhausted":
            resource_id = f"pg_conn_{target_db}"

            # 2. Enforce Circuit Breaker Safety
            if is_circuit_broken(resource_id):
                logger.error(f"Circuit breaker OPEN for {resource_id}. Too many
auto-remediations. Alerting SREs!")
                results.append({"resource": resource_id, "action": "skipped", "reason":
"circuit_breaker_open"})
                continue

            # 3. Execute Idempotent Remediation
            logger.warning(f"Executing self-healing for {alert_name} on DB: {target_db}")
            killed_connections = terminate_idle_db_connections(target_db)
            record_remediation_event(resource_id)

            results.append({
                "resource": resource_id,
                "action": "terminated_idle_connections",
                "connections_closed": killed_connections
            })

    return jsonify({"status": "processed", "results": results}), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5001)
```

Q46. How do you troubleshoot low-level Network Socket Exhaustion and File Descriptor leaks using Python's system diagnostic primitives (`psutil`, `resource`, `/proc` parsing)?

Detailed Answer:

In high-throughput Linux microservice environments, applications can fail due to running out of Linux File Descriptors (FDs). In Linux, "everything is a file": opened log streams, raw files, storage volumes, UNIX domain sockets, and active TCP connections all consume FDs allocated under the process file descriptor table.

System failure manifests in two distinct patterns:

1. Process-Level Hard/Soft Limit Breach (`RLIMIT_NOFILE`): The OS throws `OSError: [Errno 24] Too many open files` when attempting to call `accept()` or open a socket connection.

2. Socket Exhaustion (`TCP: CLOSE_WAIT` buildup): Unclosed client/server connections remain stuck in `CLOSE_WAIT` state when the remote peer initiates socket teardown (`FIN`), but the local application fails to call `socket.close()`.

SREs leverage Python to dynamically inspect process state via Linux's `/proc/<pid>/fd` virtual filesystem, analyze system calls, dynamically increase process limits (`resource.setrlimit`), and output targeted diagnostic alerts when socket accumulation indicates an unclosed connection pool leak.

Production Scenario / Practical Example:

The script below is a daemon process inspector deployed on application nodes. It scans running target Python/Node application PIDs, parses active file descriptors, categorizes network socket health states from `/proc/net/tcp`, dynamically increases `RLIMIT_NOFILE` if nearing limits, and triggers diagnostic dumps before service failure.

```
import os
import sys
import resource
import psutil
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")

def inspect_process_fd_health(target_pid: int):
    """Inspects File Descriptor limits, current usage, and socket breakdown for a target
PID."""
    try:
        proc = psutil.Process(target_pid)
    except psutil.NoSuchProcess:
        logging.error(f"PID {target_pid} does not exist.")
        return

    # Fetch limits (soft, hard)
    soft_limit, hard_limit = resource.getrlimit(resource.RLIMIT_NOFILE)

    # List all open file descriptors via /proc interface or psutil
    open_files = proc.open_files()
    connections = proc.connections(kind='inet')
    num_fds = proc.num_fds()

    logging.info(f"--- Diagnostic Report for PID {target_pid} ({proc.name()}) ---")
    logging.info(f"FD Count Usage: {num_fds} / Soft Limit: {soft_limit} (Hard Limit:
{hard_limit})")
```

```
    # Analyze socket state distribution
    socket_states = {}
    for conn in connections:
        state = conn.status
        socket_states[state] = socket_states.get(state, 0) + 1

    logging.info(f"Network Sockets State Breakdown: {socket_states}")

    # Check for File Descriptor exhaustion proximity (>80% of limit)
    usage_ratio = num_fds / soft_limit
    if usage_ratio > 0.8:
        logging.warning(f"HIGH FD USAGE DETECTED: Process is using {usage_ratio*100:.2f}% of
max FDs!")

        # Attempt Dynamic Remediation: Increase Soft Limit up to Hard Limit
        if soft_limit < hard_limit:
            new_soft_limit = min(soft_limit * 2, hard_limit)
            try:
                resource.setrlimit(resource.RLIMIT_NOFILE, (new_soft_limit, hard_limit))
                logging.info(f"SUCCESS: Dynamic bump of RLIMIT_NOFILE soft limit from
{soft_limit} -> {new_soft_limit}")
            except Exception as e:
                logging.error(f"Failed to dynamically raise RLIMIT_NOFILE limit: {e}")

    # Check for socket leak signature: excessive CLOSE_WAIT
    close_wait_count = socket_states.get(psutil.CONN_CLOSE_WAIT, 0)
    if close_wait_count > 100:
        logging.error(
            f"SOCKET LEAK DETECTED: {close_wait_count} sockets stuck in CLOSE_WAIT state. "
            "Application code is failing to explicitly call socket.close() after client
disconnected!"
        )

if __name__ == "__main__":
    current_pid = os.getpid()
    inspect_process_fd_health(current_pid)
```

Q47. How do you construct a self-healing Kubernetes Custom Operator in Python using the `kopf` framework for Stateful Application reconciliation?

Detailed Answer:

While standard Kubernetes controllers manage stateless applications via Deployments, complex stateful applications (e.g., dynamic database clusters, Vault instances, caching layers) require operational knowledge encoded into custom controllers (Operators). Implementing a Custom Operator in Python using the `kopf`

(Kubernetes Operator Python Framework) platform allows SREs to abstract complex operational runbooks into event-driven Kubernetes state reconciliation loops.

Operator Architectural Components:

1. Custom Resource Definitions (CRDs): Registering custom schema parameters in Kubernetes API (e.g., `kind: RedisCluster`).

2. Reconciliation Loop (`@kopf.on.create`, `@kopf.on.update`, `@kopf.on.timer`): Declarative logic that periodically evaluates the actual infrastructure state against the desired state defined in the CRD spec.

3. Status Subresources: Updating `.status` object keys to track health, cluster topology, and step execution markers.

4. Transient Error Handling: Raising `kopf.TemporaryError` forces the framework to retry the handler after a controlled backoff duration without marking the reconciliation as failed.

Production Scenario / Practical Example:

Below is a complete Python Custom Operator controlling a stateful database cluster deployment. It provisions secondary read-replicas, monitors their availability status, and automatically heals missing instance state if pods disappear.

```
import kopf
import os
from kubernetes import client, config

# Load active context on operator initialization
try:
    config.load_incluster_config()
except config.ConfigException:
    config.load_kube_config()

v1_apps = client.AppsV1Api()
v1_core = client.CoreV1Api()

@kopf.on.create('database.sre.io', 'v1alpha1', 'customdbclusters')
@kopf.on.update('database.sre.io', 'v1alpha1', 'customdbclusters')
def reconcile_db_cluster(spec, name, namespace, logger, **kwargs):
    """Reconciliation loop forcing actual state to match CRD desired spec."""
    desired_replicas = spec.get('replicas', 2)
    db_engine_version = spec.get('version', '14.1')

    logger.info(f"Reconciling CustomDBCluster {name} in {namespace}. Target Replicas:
{desired_replicas}")

    # 1. Define Desired StatefulSet Structure
    statefulset_manifest = {
        "apiVersion": "apps/v1",
        "kind": "StatefulSet",
```

```
        "metadata": {"name": name, "namespace": namespace},
        "spec": {
            "replicas": desired_replicas,
            "selector": {"matchLabels": {"app": name}},
            "template": {
                "metadata": {"labels": {"app": name}},
                "spec": {
                    "containers": [{
                        "name": "postgres",
                        "image": f"postgres:{db_engine_version}",
                        "env": [{"name": "POSTGRES_PASSWORD", "value": "sreSecret123"}]
                    }]
                }
            }
        }
    }

    # 2. Check current state and adopt/update child resources idempotently
    try:
        existing_sts = v1_apps.read_namespaced_stateful_set(name=name, namespace=namespace)
        current_replicas = existing_sts.spec.replicas

        # Patch StatefulSet if desired state diverges
        if current_replicas != desired_replicas:
            logger.info(f"Scaling STS {name} from {current_replicas} to {desired_replicas}")
            v1_apps.patch_namespaced_stateful_set(
                name=name,
                namespace=namespace,
                body={"spec": {"replicas": desired_replicas}}
            )

    except client.exceptions.ApiException as e:
        if e.status == 404:
            # Resource doesn't exist, create it
            logger.info(f"Creating new StatefulSet for {name}")
            kopf.adopt(statefulset_manifest) # Auto-binds owner references for K8s garbage
collection
            v1_apps.create_namespaced_stateful_set(namespace=namespace,
body=statefulset_manifest)
        else:
            # Transient cluster network failure, schedule backoff retry
            raise kopf.TemporaryError(f"Kubernetes API Error: {e}", delay=15)

    return {"phase": "Ready", "active_nodes": desired_replicas}

@kopf.on.timer('database.sre.io', 'v1alpha1', 'customdbclusters', interval=30.0)
def verify_cluster_health(name, namespace, logger, **kwargs):
    """Periodic health-check timer driving self-healing behavior."""
```

```
    try:
        pods = v1_core.list_namespaced_pod(namespace=namespace, label_selector=f"app={name}")
        unhealthy_pods = [p.metadata.name for p in pods.items if p.status.phase != "Running"]

        if unhealthy_pods:
            logger.warning(f"Unhealthy DB pods detected in cluster {name}: {unhealthy_pods}")
            # Self-healing logic could delete corrupted pods to force StatefulSet rebuilds
    except Exception as e:
        logger.error(f"Health audit check failed: {e}")
```

Q48. How do you implement Zero-Downtime dynamic configuration reloads and POSIX Signal handling (`SIGHUP`, `SIGTERM`) in enterprise Python daemons?

Detailed Answer:

In Kubernetes environments, pod terminations, rolling updates, and autoscaling triggers send standard POSIX termination signals (`SIGTERM`) to PID 1 inside the container workspace. If a Python service fails to handle POSIX signals correctly:

1. It ignores `SIGTERM`, resulting in forced process killing via `SIGKILL` (exit code 137) after the standard Kubernetes `terminationGracePeriodSeconds` (default 30s) expires.

2. In-flight transactions, pending log/metric flushes, and active database connection state corruption occurs.

3. Configuration updates force a full pod restart cycle, impacting latency SLAs.

To build zero-downtime microservices and SRE daemons:

- **`SIGTERM` / `SIGINT` Handling**: Intercept the signal, deregister the instance from service discovery endpoints, complete currently running tasks, close open socket pools cleanly, and terminate gracefully with exit code `0`.
- **`SIGHUP` Dynamic Reloading**: Intercept `SIGHUP` (Signal Hang Up) to trigger live configuration reloads from local disk without restarting the underlying process or severing active TCP listener sockets.

Production Scenario / Practical Example:

The following script demonstrates a daemon that listens for system signals. On receiving `SIGHUP`, it reloads configuration dynamically without dropping processing state. On receiving `SIGTERM`, it sets a shutdown status flag, drains running worker threads, flushes output buffers, and exits cleanly.

```
import signal
import sys
import time
import json
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
```

```
class GracefulDaemon:
    def __init__(self, config_path: str):
        self.config_path = config_path
        self.running = True
        self.in_flight_jobs = 0
        self.config = {}

        # Register POSIX Signal Handlers
        signal.signal(signal.SIGHUP, self._handle_sighup)
        signal.signal(signal.SIGTERM, self._handle_sigterm)
        signal.signal(signal.SIGINT, self._handle_sigterm)

        self._load_config()

    def _load_config(self):
        """Loads or hot-reloads configuration settings."""
        try:
            # Simulating reading JSON config file from disk
            logging.info(f"Loading configuration state from {self.config_path}...")
            # For demonstration, mock config parsing:
            self.config = {"max_workers": 4, "timeout": 30, "reloaded_at": time.time()}
            logging.info(f"Configuration successfully active: {self.config}")
        except Exception as e:
            logging.error(f"Failed to reload config: {e}. Retaining old config.")

    def _handle_sighup(self, signum, frame):
        """Dynamic Reload Handler triggered without process restart."""
        logging.warning("SIGHUP received! Initiating live dynamic configuration reload...")
        self._load_config()

    def _handle_sigterm(self, signum, frame):
        """Graceful Shutdown Handler triggered by Kubernetes preStop/SIGTERM."""
        signal_name = "SIGTERM" if signum == signal.SIGTERM else "SIGINT"
        logging.warning(f"{signal_name} received! Initiating graceful shutdown sequence...")

        # 1. Stop accepting new requests/jobs
        self.running = False

        # 2. Complete draining in-flight operations
        logging.info(f"Draining {self.in_flight_jobs} active worker tasks...")
        while self.in_flight_jobs > 0:
            logging.info(f"Waiting for {self.in_flight_jobs} jobs to complete...")
            time.sleep(0.5)

        # 3. Clean up resource handles
        self._cleanup_resources()
        logging.info("Graceful shutdown finished complete. Exiting clean with code 0.")
        sys.exit(0)
```

```
    def _cleanup_resources(self):
        """Closes open sockets, connection pools, and flushes log buffers."""
        logging.info("Flushing log buffers, terminating database connections...")

    def run_work_loop(self):
        logging.info("Daemon active and processing tasks. Target PID: " + str(os.getpid()))
        while self.running:
            # Simulate work
            self.in_flight_jobs += 1
            time.sleep(1) # Processing task...
            self.in_flight_jobs -= 1

            time.sleep(0.5)

if __name__ == "__main__":
    import os
    daemon = GracefulDaemon(config_path="/etc/daemon/config.json")
    daemon.run_work_loop()
```

Q49. How do you instrument a Python application using OpenTelemetry to capture custom distributed traces, dynamic span attributes, and contextual SRE telemetry?

Detailed Answer:

Distributed tracing is essential for identifying microservice performance bottlenecks, cross-network latency, and cascading failure pathways across complex distributed system architectures. OpenTelemetry (OTel) provides a vendor-agnostic framework to generate, instrument, and export trace telemetry.

Key Architectural Components in OpenTelemetry Python:

1. Tracer Provider (`TracerProvider`): A stateful factory holding tracing configuration, samplers (e.g., `ParentBased` or `TraceIdRatioBasedSampler`), and global resource attributes (e.g., `service.name`, `deployment.environment`).

2. Span Processors (`BatchSpanProcessor`): Asynchronously aggregates generated spans in memory and exports them in batches via gRPC/HTTP OTLP exporters, minimizing CPU impact on application event loops.

3. Context Propagation (`W3C TraceContext`): Injects and extracts tracing headers (`traceparent`, `tracestate`) across HTTP/gRPC boundaries to preserve trace context continuity between upstream callers and downstream dependencies.

4. Custom Span Annotations: Injecting high-value operational metadata (e.g., database query parameters, custom SRE performance metrics, exception stack traces via `span.record_exception()`).

Production Scenario / Practical Example:

Below is an enterprise service instrumented with OpenTelemetry. It creates nested custom spans, injects

business context, records failure states, and propagates the trace context over outgoing HTTP calls.

```
import requests
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.resources import Resource
from opentelemetry.trace.propagation.tracecontext import TraceContextTextMapPropagator
from opentelemetry.trace import Status, StatusCode

# 1. Initialize Global Tracer Provider with Service Attributes
resource = Resource.create({"service.name": "payment-processing-engine", "env":
"production"})
provider = TracerProvider(resource=resource)

# 2. Configure OTLP gRPC Exporter to send traces to OpenTelemetry Collector
otlp_exporter = OTLPSpanExporter(endpoint="otel-collector.monitoring.svc.cluster.local:4317",
insecure=True)
provider.add_span_processor(BatchSpanProcessor(otlp_exporter))

# Register provider globally
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("sre.diagnostics.tracer")

def execute_downstream_http_call(payload: dict):
    """Executes external call while propagating trace context across HTTP headers."""
    url = "https://api.stripe.com/v1/charges"
    headers = {"Content-Type": "application/json"}

    # Inject W3C TraceContext headers into outgoing request
    TraceContextTextMapPropagator().inject(headers)

    with tracer.start_as_current_span("stripe_charge_api_call") as span:
        span.set_attribute("http.method", "POST")
        span.set_attribute("http.url", url)

        try:
            # Mocking outbound request
            response = requests.post(url, json=payload, headers=headers, timeout=2.0)
            span.set_attribute("http.status_code", response.status_code)
            return response.status_code
        except Exception as e:
            # Record exception events and mark span status as ERROR
            span.record_exception(e)
            span.set_status(Status(StatusCode.ERROR, description=str(e)))
            raise
```

```
def process_payment_transaction(transaction_id: str, amount: float):
    """Main execution path wrapped inside primary trace span."""
    with tracer.start_as_current_span("process_payment_transaction") as parent_span:
        parent_span.set_attribute("transaction.id", transaction_id)
        parent_span.set_attribute("transaction.amount", amount)

        # Internal computational step span
        with tracer.start_as_current_span("validate_account_balance") as sub_span:
            sub_span.set_attribute("account.status", "active")
            # Simulate DB/Cache check

        # Downstream execution step
        try:
            execute_downstream_http_call({"id": transaction_id, "amount": amount})
        except Exception as err:
            parent_span.set_attribute("error.remediation_required", True)

if __name__ == "__main__":
    process_payment_transaction("tx_9928341", 450.50)
```

Q50. How do you design and execute automated Chaos Engineering experiments in Python to validate system resilience under forced Network Latency and Interface Degradation?

Detailed Answer:

Chaos Engineering is the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production. Building Chaos runners using Python enables SREs to automate steady-state hypothesis testing, fault injection execution, failure monitoring, and immediate rollback procedures.

Key Architecture of a Chaos Experimentation Engine:

1. Steady-State Hypothesis Validation: Querying metrics APIs (e.g., Prometheus) *before* fault injection to ensure the target ecosystem is healthy (e.g., Success Rate $> 99.9\%$, Latency $p99 < 200\text{ms}$).

2. Blast-Radius Containment and Fault Injection: Interacting with target node environments or container namespaces via `subprocess` or SSH using Linux traffic control utilities (`tc qdisc`) or Linux network rules (`iptables`).

3. Safety Guardrails and Automatic Rollback: Context managers (`try...finally`) ensuring that regardless of whether the experiment succeeds, unhandled script exceptions occur, or SLAs break, the fault injection is immediately reverted.

Production Scenario / Practical Example:

The Python script below executes a Chaos Engineering experiment on a microservice node. It measures baseline health via Prometheus, uses Linux `tc` (Traffic Control) to inject 300ms network latency and

10% packet loss into a network interface, checks health continuously, and auto-rolls back if error rates breach steady-state thresholds.

```
import subprocess
import time
import logging
import requests

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")

PROMETHEUS_URL = "http://prometheus.monitoring.svc.cluster.local:9090"
TARGET_INTERFACE = "eth0"

class ChaosNetworkExperiment:
    def __init__(self, interface: str, added_latency_ms: int = 300, packet_loss_percent: int
= 10):
        self.interface = interface
        self.added_latency_ms = added_latency_ms
        self.packet_loss_percent = packet_loss_percent

    def check_steady_state_health(self) -> bool:
        """Queries Prometheus to ensure current HTTP error rate is below 1% threshold."""
        query = 'sum(rate(http_requests_total{status=~"5.."}[2m])) /
sum(rate(http_requests_total[2m]))'
        try:
            res = requests.get(f"{PROMETHEUS_URL}/api/v1/query", params={"query": query},
timeout=3.0)
            data = res.json()
            results = data.get("data", {}).get("result", [])

            if not results:
                return True # No errors recorded

            error_rate = float(results[0]["value"][1])
            logging.info(f"Current System Error Rate Metric: {error_rate*100:.3f}%")
            return error_rate < 0.01
        except Exception as e:
            logging.error(f"Failed to query Prometheus steady-state metrics: {e}")
            return False

    def inject_network_fault(self):
        """Uses tc (traffic control) netem to add packet latency and loss."""
        logging.warning(f"INJECTING FAULT: Adding {self.added_latency_ms}ms latency &
{self.packet_loss_percent}% packet loss to {self.interface}")
        cmd = [
            "sudo", "tc", "qdisc", "add", "dev", self.interface, "root", "netem",
            "delay", f"{self.added_latency_ms}ms",
            "loss", f"{self.packet_loss_percent}%"
```

```
        ]
        subprocess.run(cmd, check=True)

    def rollback_network_fault(self):
        """Clears all tc qdisc traffic shaping rules from the target network interface."""
        logging.info(f"ROLLING BACK FAULT: Clearing tc traffic rules on {self.interface}...")
        cmd = ["sudo", "tc", "qdisc", "del", "dev", self.interface, "root"]
        subprocess.run(cmd, check=False) # Ignore errors if already cleared

    def run_experiment(self, duration_seconds: int = 30):
        logging.info("--- Starting Chaos Engineering Experiment ---")

        # 1. Validate Initial Steady-State
        if not self.check_steady_state_health():
            logging.error("ABORTING EXPERIMENT: System is not in healthy steady-state
baseline!")
            return

        try:
            # 2. Inject Fault
            self.inject_network_fault()

            # 3. Observe metrics during blast window
            start_time = time.time()
            while time.time() - start_time < duration_seconds:
                time.sleep(5)
                # Verify SLA safety limits during active chaos execution
                if not self.check_steady_state_health():
                    logging.critical("EMERGENCY ABORT: Error rate threshold breached during
Chaos run! Triggering immediate rollback.")
                    break

        except Exception as ex:
            logging.error(f"Unexpected exception during chaos execution: {ex}")
        finally:
            # 4. Mandatory Rollback Guardrail (Executes regardless of failure)
            self.rollback_network_fault()
            logging.info("Chaos experiment run completed and environment reverted.")

if __name__ == "__main__":
    # Ensure running on a designated staging/chaos node with proper permissions
    experiment = ChaosNetworkExperiment(interface="eth0", added_latency_ms=250,
packet_loss_percent=5)
    experiment.run_experiment(duration_seconds=20)
```